Spectral Methods for Binary Neural Networks

Training 1-Bit Language Models via Boolean Fourier Analysis

A Thesis Proposal

[Author Name]

[Department and Institution]

February 2026

Revised Draft — Incorporating Committee Feedback

Abstract

Recent work on 1-bit large language models (BitNet and its variants) has demonstrated that neural networks with binary or ternary weights can achieve performance competitive with full-precision models. However, all existing approaches rely on full-precision shadow weights during training, floating-point scaling factors, and higher-precision activations—undermining the claim of true 1-bit computation. We propose a spectral lens for binary neural networks, grounded in Boolean Fourier analysis and the Walsh–Hadamard transform. Our key insight is that the combinatorial explosion inherent in searching over 2^n binary weight configurations can be tamed by viewing the optimization through the spectral domain, where practical Boolean functions often have sparse, low-degree representations. We develop a framework in which network layers are parameterized by their Walsh–Fourier coefficients, providing a better inductive bias and a principled set of diagnostics. Our primary architecture—the *Spectral Binary Transformer* (SBT)—is built entirely on the row-wise spectral parameterization. The Hadamard-structured approach (Section 4.4) is included as an alternative/ablation if the row-wise spectral concentration hypothesis fails. We present preliminary theoretical results, including an approximation bound for degree-truncated spectral parameterizations and a formal analysis of normalization propagation in Hadamard-structured networks. We also propose a pilot empirical study of spectral concentration in existing binary weight matrices. This work bridges two communities that have developed largely in isolation: the Boolean function analysis community in theoretical computer science and the quantized neural network community in machine learning.

Contents

1 Introduction **4**
1.1 Motivation . . . 4
1.2 The Precision Leakage Problem . . . 4
1.3 Our Thesis: Boolean Fourier Analysis as a Training Framework . . . 5
1.4 Related Work . . . 6
1.4.1 Binary and Ternary Neural Networks . . . 6
1.4.2 Boolean Fourier Analysis . . . 7
1.4.3 Walsh–Hadamard Transform in Neural Networks . . . 7
1.4.4 Structured Matrices in Deep Learning . . . 7
1.5 Open Problems and Contributions . . . 8

2 Mathematical Background **9**
2.1 The Boolean Hypercube and Functions Thereon . . . 9
2.2 Fourier Expansion on the Boolean Hypercube . . . 9
2.3 Parseval's Theorem and the Energy Constraint . . . 10
2.4 Spectral Concentration and the LMN Theorem . . . 11
2.5 Influence, Noise Sensitivity, and Hypercontractivity . . . 11
2.5.1 Influence of Variables . . . 11
2.5.2 Noise Sensitivity and Stability . . . 12
2.5.3 The Bonami–Beckner Inequality . . . 12
2.6 The Walsh–Hadamard Transform . . . 12
2.7 The Sign Activation in the Fourier Domain . . . 13
2.8 Multi-Layer Spectral Composition . . . 13
2.9 Group-Theoretic Perspective . . . 14

3 Theoretical Contributions **16**
3.1 Approximation by Degree-Truncated Spectral Parameterization . . . 16
3.2 Normalization Propagation in Hadamard Networks . . . 17
3.3 Formal Conjectures . . . 19

4 Architecture and Methods **21**
4.1 Target Architecture: Proof-of-Concept Scope . . . 21
4.2 Module 1: Spectral Layer Parameterization . . . 26
4.2.1 Row-Wise Spectral Decomposition . . . 26
4.2.2 The Column-Indexing Problem . . . 27
4.3 Module 2: Spectral-to-Binary Projection . . . 28
4.3.1 Hard Sign Projection . . . 29
4.3.2 Randomized Rounding . . . 29

4.3.3 Spectral Norm Projection 29
4.3.4 Sparse Spectral Layers (Variant) 29
4.4 Alternative Approach: Hadamard-Structured Layers 29
4.5 Module 3: Spectral Attention Mechanism 30
4.6 Module 4: Training Algorithm 32
4.7 Module 5: Spectral Regularization 33
4.8 A Worked Toy Example: Learning the Majority Function 34

5 Pilot Study: Spectral Concentration in Existing Binary Weights 37
5.1 Protocol 37
5.2 Expected Outcomes and Implications 38
5.3 Computational Cost 39

6 Experimental Plan 40
6.1 Datasets and Tasks 40
6.2 Baselines 40
6.3 Metrics 40
6.4 Ablation Studies 41
6.5 Implementation Roadmap 41
6.6 Success Criteria 41
6.7 Known Risks and Mitigation 42

1 Introduction

1.1 Motivation

The computational and energy costs of large language models have become a central concern in modern artificial intelligence. A single training run for a frontier model can consume megawatt-hours of electricity and require thousands of specialized accelerators [Patterson et al., 2021]. At inference time, the memory bandwidth required to shuttle 16-bit floating-point weights between memory and compute units is often the primary bottleneck, not arithmetic itself [Kim et al., 2023]. These constraints have motivated intense interest in weight quantization—reducing the precision of model parameters from 16 or 32 bits down to 8, 4, or even fewer bits per weight.

The extreme endpoint of this trajectory is the *1-bit neural network*: a model in which every weight is drawn from the set $\{-1, +1\}$. Such a network replaces all multiply-accumulate operations with additions and subtractions, eliminates the need for floating-point arithmetic at inference, and compresses the model by a factor of 16× relative to FP16. The potential impact is enormous: 1-bit models could run on commodity CPUs, embedded devices, and edge hardware with minimal power consumption, democratizing access to capable language models.

Yet despite the appeal, no one has achieved a *truly* 1-bit language model. The state of the art—Microsoft Research's BitNet family [Wang et al., 2023, Ma et al., 2024b]—comes close but relies on several full-precision crutches that compromise the purity of the binary paradigm. Understanding why these crutches are needed, and how they might be removed, requires a mathematical framework that the quantization community has largely overlooked: the Fourier analysis of Boolean functions.

1.2 The Precision Leakage Problem

We identify three specific ways in which current "1-bit" approaches leak precision beyond the binary constraint.

Shadow weights. During training, BitNet maintains full-precision (FP16 or FP32) copies of all weights. The binary weights used in the forward pass are obtained by applying the sign function to these latent continuous weights. Gradients are computed with respect to the latent weights using the straight-through estimator (STE), which approximates the derivative of the sign function as 1. The latent weights are the true learned parameters; the binary weights are merely a projection applied at each forward pass. This means training is fundamentally a continuous optimization problem masquerading as a discrete one. We return to this point in the next paragraph: our approach also uses continuous "shadow" state during training (in the spectral domain) and does not claim to remove

it; our contributions are a different parameterization, the elimination of learned scaling factors, and a principled diagnostic framework.

Scaling factors. Each layer (or sub-layer) in BitNet is equipped with a learned floating-point scaling factor α that multiplies the output of the binary linear transformation. Without these factors, the activations would have magnitudes determined entirely by the dimension of the layer, with no mechanism for the network to modulate signal strength across layers. The scaling factors constitute a significant hidden parameter budget that operates at full precision.

Activation precision. While the weights are binary, the activations flowing between layers are typically quantized to 8 bits (or kept at full precision during training). Since the output of a binary linear layer is an integer in the range $[-n, +n]$ where n is the input dimension, the activations carry $\log_2(2n + 1)$ bits of information per element—far more than 1 bit.

We do not claim these design choices are flawed—they are pragmatic and effective. But they leave open the fundamental question: *Is it possible to train a neural network that is purely binary, end to end, without recourse to full-precision arithmetic?* And if so, what mathematical framework would guide such an effort?

1.3 Our Thesis: Boolean Fourier Analysis as a Training Framework

We propose that the Fourier analysis of Boolean functions—a mature branch of theoretical computer science with deep connections to combinatorics, learning theory, and harmonic analysis—provides the missing mathematical foundation for truly 1-bit neural networks.

The central observation is this: in the *weight space* of a binary network, we face an unstructured combinatorial search over 2^n configurations. But in the *spectral space* (the Walsh–Fourier domain), the same functions admit structured, often sparse representations that can be manipulated with continuous optimization. The Walsh–Hadamard transform—the natural Fourier transform on the Boolean hypercube—provides the bridge between these two views. Its basis functions are themselves ± 1-valued, its computation requires only additions and subtractions, and it satisfies all the algebraic properties (Parseval's identity, convolution theorem) that make classical Fourier methods powerful.

We develop this observation into a concrete research program. We define a neural network architecture—the *Spectral Binary Transformer* (SBT)—whose layers are parameterized in the Walsh–Fourier domain. We describe training algorithms that optimize spectral coefficients continuously and project to valid Boolean functions. We lay out a detailed experimental plan for evaluating this approach, including specific benchmarks,

metrics, baselines, and diagnostic tools.

A note on scope and claims. We wish to be precise about what we are and are not claiming. Our training procedure, like BitNet's, maintains continuous parameters (spectral coefficients) that serve as "shadow" state during training and are discarded at inference. We do not claim to eliminate the need for continuous optimization during training—this remains an open and likely very hard problem. Rather, our contributions are:

(i) **A better inductive bias:** The spectral parameterization encodes structural knowledge about Boolean functions (degree concentration, Parseval normalization) that raw weight-domain STE does not.

(ii) **Replacement of learned scaling factors with fixed constants:** The per-layer scaling factor α in BitNet is a learned floating-point parameter that modulates activation variance. In our framework, the corresponding factor is the analytically determined constant $1/\sqrt{d}$ (Proposition 3.3), which serves the same variance-control role but requires no learning and no floating-point parameter storage at inference.

(iii) **A principled theoretical framework:** Boolean Fourier analysis provides tools (influence, noise sensitivity, hypercontractivity) for *understanding* binary networks, not just training them.

The ultimate goal of *purely binary training* (no continuous parameters even during training) is a long-term aspiration that we identify as a direction for future work but do not promise to solve here.

1.4 Related Work

1.4.1 Binary and Ternary Neural Networks

The idea of training neural networks with binary weights dates to the early connectionism literature, but modern interest was reignited by BinaryConnect [Courbariaux et al., 2015], which introduced the straight-through estimator (STE) for binary weight training [Bengio et al., 2013]. Binarized Neural Networks [Hubara et al., 2016] extended this to both weights and activations, and XNOR-Net [Rastegari et al., 2016] demonstrated practical ImageNet-scale binary networks. DoReFa-Net [Zhou et al., 2016] generalized to low-bit weights, activations, and gradients. Bi-Real Net [Liu et al., 2018] improved performance with additional residual connections. Helwegen et al. [2019] challenged the conventional latent-weight view, arguing that binarized optimization dynamics differ fundamentally from their continuous counterparts; Yin et al. [2019] provided a theoretical analysis of the STE's convergence properties. The BitNet line of work [Wang et al., 2023] scaled binary

(and later ternary [Ma et al., 2024b]) weight training to the Transformer architecture and language modeling, achieving the best results to date. FBI-LLM [Ma et al., 2024a] trains fully binarized LLMs from scratch via autoregressive distillation. All of these methods rely on the STE and full-precision shadow weights.

1.4.2 Boolean Fourier Analysis

The Fourier analysis of Boolean functions has been a central tool in theoretical computer science since the foundational work of Kahn et al. [1988] on the influence of variables. The textbook treatment by O'Donnell [2014] provides a comprehensive reference. Key results that inform our work include the Linial–Mansour–Nisan theorem [Linial et al., 1989] on the low-degree spectral concentration of AC^0 circuits, the Kushilevitz–Mansour algorithm [Kushilevitz and Mansour, 1991] for learning Fourier-sparse Boolean functions, the Bonami–Beckner hypercontractivity inequality [Bonami, 1970, Beckner, 1975] and its applications to noise sensitivity, and the body of work on the learnability of Boolean function classes via spectral methods [Mansour, 1994].

1.4.3 Walsh–Hadamard Transform in Neural Networks

Pan et al. [2021, 2022] introduced Walsh–Hadamard transform layers as replacements for convolutional layers in vision networks, achieving significant parameter savings. Their work demonstrates the computational viability of WHT-based layers but does not address the binary weight constraint or language modeling. On the algorithmic side, Alman [2022] gives faster algorithms for the Walsh–Hadamard transform over finite fields using lookup tables, which may inform future hardware-aware implementations. The Walsh–Hadamard Neural Operator [Cavallazzi et al., 2025] uses WHT-based spectral processing for solving PDEs with discontinuous coefficients, demonstrating the advantage of Walsh bases for piecewise-constant structure. Raviv et al. [2021] applied Boolean Fourier analysis to neural network robustness, developing "Fourier stabilization" of individual neurons.

1.4.4 Structured Matrices in Deep Learning

Hadamard matrices and other structured transforms have been used to reduce parameter counts and accelerate computation in neural networks. Notable examples include the Fastfood transform [Le et al., 2013] for kernel approximation, the structured spinners of Choromanski et al. [2017] for random feature methods, butterfly factorizations for learning fast linear transforms [Dao et al., 2019], and the Monarch matrices of Dao et al. [2022] which generalize them. Neural networks also exhibit a spectral bias toward low-frequency functions during training in the classical (sinusoidal) Fourier sense [Rahaman et al., 2019]; while the domain differs, this provides an analogy supporting our hypothesis that trained binary weights concentrate on low-degree Boolean Fourier coefficients. Our

work differs from these lines in that we use structured transforms not merely for efficiency but as the *native representation* of binary-valued weight matrices.

1.5 Open Problems and Contributions

This thesis addresses the following open problems.

Problem 1 (Spectral Parameterization)**.** Does parameterizing binary network layers in the Walsh–Fourier domain yield a training landscape that is more amenable to optimization than the discrete weight space, and does it produce a cleaner binary inference artifact (no learned scaling factors)?

Problem 2 (Spectral Sparsity in Practice)**.** Do the weight matrices learned by practical binary language models exhibit low-degree spectral concentration in the Boolean Fourier sense?

Problem 3 (Multi-Layer Spectral Composition)**.** How does the spectral structure of binary weight functions interact with sign activations across multiple layers, and what are the implications for trainability and expressivity?

Our contributions toward these problems are:

(a) A formal framework connecting Boolean Fourier analysis to binary neural network training, with precise claims about what is and is not gained relative to existing approaches (Section 1.3).

(b) Preliminary theoretical results, including an approximation bound for degree-truncated spectral parameterization and a normalization propagation analysis for Hadamard-structured networks (Section 3).

(c) A concrete architecture (SBT) that instantiates this framework, with a detailed design of each module (Section 4).

(d) A pilot empirical study protocol for testing the core spectral concentration assumption on existing binary weights (Section 5).

(e) A comprehensive experimental plan including baselines, metrics, and diagnostic procedures (Section 6).

2 Mathematical Background

This chapter provides a self-contained treatment of the mathematical tools underlying our approach. We assume familiarity with linear algebra, basic probability, and the Transformer architecture, but develop the Boolean Fourier analysis and Walsh–Hadamard theory from first principles.

2.1 The Boolean Hypercube and Functions Thereon

Definition 2.1. Let $\Omega_n = \{-1, +1\}^n$ denote the *Boolean hypercube* of dimension n. We equip Ω_n with the uniform probability measure, so each vertex $x \in \Omega_n$ has probability 2^{-n}.

A *Boolean function* is a mapping $f\colon \Omega_n \to \{-1, +1\}$. More generally, we consider *pseudo-Boolean functions* $f\colon \Omega_n \to \mathbb{R}$.

The space of all functions $f\colon \Omega_n \to \mathbb{R}$ forms a 2^n-dimensional real vector space equipped with the inner product

$$\langle f, g\rangle = \mathbb{E}_x[f(x)\, g(x)] = 2^{-n} \sum_{x\in\Omega_n} f(x)\, g(x) \tag{1}$$

and the associated L^2 norm $\|f\|_2 = \sqrt{\langle f, f\rangle}$. For Boolean functions, we always have $\|f\|_2 = 1$.

2.2 Fourier Expansion on the Boolean Hypercube

Definition 2.2. For each subset $S \subseteq [n] = \{1, 2, \ldots, n\}$, define the *parity function* (or *character*)

$$\chi_S(x) = \prod_{i\in S} x_i, \tag{2}$$

with the convention that $\chi_\emptyset(x) = 1$ for all x.

These 2^n functions form an orthonormal basis for the space of all functions on Ω_n:

$$\langle \chi_S, \chi_T\rangle = \delta_{ST}. \tag{3}$$

Critically, each χ_S is itself a Boolean function—it takes values in $\{-1, +1\}$. This is a distinctive feature of the Boolean Fourier basis compared to the classical sinusoidal basis: *the basis elements live in the same* $\{\pm 1\}$ *alphabet as the functions being analyzed.*

Theorem 2.3 (Fourier Expansion)**.** *Every function* $f\colon \Omega_n \to \mathbb{R}$ *has a unique expansion*

$$f(x) = \sum_{S\subseteq[n]} \widehat{f}(S)\, \chi_S(x), \tag{4}$$

where the Fourier coefficients *are*

$$\widehat{f}(S) = \langle f, \chi_S \rangle = \mathbb{E}_x \left[f(x) \prod_{i \in S} x_i \right] = 2^{-n} \sum_{x \in \Omega_n} f(x) \prod_{i \in S} x_i. \tag{5}$$

Definition 2.4. The cardinality $|S|$ is called the *degree* of the monomial χ_S. We say f has *Fourier degree at most* d if $\widehat{f}(S) = 0$ for all $|S| > d$.

2.3 Parseval's Theorem and the Energy Constraint

Theorem 2.5 (Parseval's Identity)**.** *For any* $f \colon \Omega_n \to \mathbb{R}$*,*

$$\|f\|_2^2 = \sum_{S \subseteq [n]} \widehat{f}(S)^2. \tag{6}$$

For a Boolean function $f \colon \Omega_n \to \{-1, +1\}$, we have $\|f\|_2^2 = 1$, hence:

$$\sum_{S \subseteq [n]} \widehat{f}(S)^2 = 1. \tag{7}$$

Remark 2.6. This is a fundamental structural constraint: the Fourier coefficients of any Boolean function necessarily lie on the unit sphere in $\mathbb{R}^{2^n}$ (in the 2^n-dimensional coefficient space, not the n-dimensional input space or d-dimensional weight space). Two complementary normalization principles allow us to replace BitNet's learned scaling factors with fixed constants:

(i) *Parseval (parameter-level):* $\sum_S \widehat{f}(S)^2 = 1$ constrains the spectral coefficients to unit L_2 norm, ensuring the parameterization is inherently calibrated regardless of the specific Boolean function represented.

(ii) *CLT (activation-level):* For *any* binary weight matrix $W \in \{-1, +1\}^{d \times d}$ with uniform binary input, the pre-activation entries converge to $\mathcal{N}(0, 1)$ after dividing by $\sqrt{d}$. For Hadamard-structured layers this scaling is additionally an exact isometry (Proposition 3.3); for general binary matrices (the spectral approach) it preserves variance in expectation (Remark 3.6). In both cases, the required scaling is a fixed constant known at initialization, not a learned parameter.

Together, these yield a normalization scheme where the coefficient energy is structurally bounded and the forward-pass variance is analytically predictable, eliminating the need for the per-layer learned α that BitNet requires.

2.4 Spectral Concentration and the LMN Theorem

A central theme in Boolean Fourier analysis is that "simple" functions have their spectral mass concentrated on low-degree coefficients.

Theorem 2.7 (Linial–Mansour–Nisan, 1989)**.** *Let $f\colon \{-1,+1\}^n \to \{-1,+1\}$ be computable by a Boolean circuit of depth D and size M. Then for every $k \geq 0$:*

$$\sum_{\substack{S\subseteq[n]\\|S|>k}} \widehat{f}(S)^2 \;\leq\; 2M \cdot \exp\left(-\frac{k^{1/(D-1)}}{2}\right). \tag{8}$$

For bounded-depth circuits (AC^0), the spectral weight is essentially concentrated on degrees up to $(\log M)^c$ for a constant c depending on the depth.

Remark 2.8 (Significance and limitations for binary networks)*.* The LMN theorem guarantees spectral concentration for functions with low circuit complexity. We do *not* claim that trained weight matrices necessarily correspond to AC^0 functions—this is an empirical question we address in Section 5. However, several lines of evidence suggest spectral concentration may hold more broadly: (i) trained neural networks exhibit strong implicit regularization toward "simple" functions in the sense of low norm or low rank [Neyshabur et al., 2015]—a distinct notion from low Boolean Fourier degree, but one that, for LTFs, implies bounded total influence and hence bounded spectral tails (see the Evidence paragraph in Section 3.3); (ii) the functions that are *useful* for language modeling (capturing statistical patterns in text) are unlikely to resemble random Boolean functions, which have flat spectra; (iii) linear threshold functions, which are the building blocks of binary networks, are known to be spectrally concentrated (Section 2.7). The pilot study in Section 5 is designed to test this assumption directly.

2.5 Influence, Noise Sensitivity, and Hypercontractivity

2.5.1 Influence of Variables

Definition 2.9. The *influence* of variable i on a Boolean function f is

$$\mathrm{Inf}_i[f] = \mathbb{P}[f(x) \neq f(x^{\oplus i})] = \sum_{S\ni i} \widehat{f}(S)^2, \tag{9}$$

where $x^{\oplus i}$ denotes x with the i-th coordinate flipped. The *total influence* is

$$\mathbf{I}[f] = \sum_{i=1}^{n} \mathrm{Inf}_i[f] = \sum_{S\subseteq[n]} |S| \cdot \widehat{f}(S)^2. \tag{10}$$

In the context of binary neural networks, the total influence governs gradient informa-

tion flow: a layer whose Boolean function has low total influence is relatively insensitive to input perturbations, aiding stability but potentially impeding learning.

2.5.2 Noise Sensitivity and Stability

Definition 2.10. For $\rho \in [0, 1]$, the *noise stability* of f at correlation ρ is

$$\mathrm{Stab}_\rho[f] = \sum_{S \subseteq [n]} \rho^{|S|} \widehat{f}(S)^2. \tag{11}$$

For our setting, noise sensitivity characterizes how errors in the binary weights propagate through the network. A layer with noise-stable Boolean functions will be robust to occasional bit flips in its weight matrix.

2.5.3 The Bonami–Beckner Inequality

Theorem 2.11 (Bonami–Beckner Hypercontractivity)**.** *For $f \colon \Omega_n \to \mathbb{R}$ and $1 \le p \le q$ with $\rho \le \sqrt{(p-1)/(q-1)}$:*

$$\|T_\rho f\|_q \le \|f\|_p \,, \tag{12}$$

where T_ρ is the Bonami–Beckner noise operator $T_\rho f(x) = \sum_S \rho^{|S|} \widehat{f}(S)\, \chi_S(x)$.

A direct corollary is the *level-k inequality*: for Boolean f,

$$\sum_{|S|=k} \widehat{f}(S)^2 \le \left(\frac{e \cdot \mathbf{I}[f]}{k} \right)^k. \tag{13}$$

This bounds the spectral weight at any fixed degree in terms of the total influence, providing a quantitative link between sensitivity and spectral structure. We use this inequality directly in our theoretical results (Section 3).

2.6 The Walsh–Hadamard Transform

Definition 2.12. The Hadamard matrix of order 2^n is defined recursively:

$$H_1 = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad H_n = H_1 \otimes H_{n-1}, \tag{14}$$

where $\otimes$ denotes the Kronecker product.

The matrix H_n is $2^n \times 2^n$, real, symmetric, orthogonal up to scaling ($H_n H_n^\top = 2^n I$), and every entry is ± 1.

Key properties.

(i) **Self-inverse (up to scaling).** $H_n^2 = 2^n I$.

(ii) **No multiplications.** For binary or integer inputs, the FWHT computes $H_n v$ using only additions and subtractions. (During training, spectral coefficients are continuous, so the inverse transform uses real-valued scalar multiplication, but the forward transform at inference is multiplication-free.)

(iii) **Convolution theorem.** Dyadic convolution in the spatial domain corresponds to pointwise multiplication in the WHT domain.

(iv) **Inherent normalization.** For a Boolean function f represented as a vector in $\{-1,+1\}^{2^n}$, the WHT coefficients satisfy $\sum \widehat{f}(S)^2 = 1$ (normalized convention).

2.7 The Sign Activation in the Fourier Domain

The sign function $\mathrm{sgn}\colon \mathbb{R} \to \{-1,+1\}$ is the natural activation for a binary network. When applied to a weighted sum of binary inputs, $\mathrm{sgn}(\sum_i w_i x_i)$ computes a *linear threshold function* (LTF).

Theorem 2.13 (Chow's Theorem; see O'Donnell [2014, Chapter 5])**.** *A linear threshold function is uniquely determined among all linear threshold functions by its degree-0 and degree-1 Fourier coefficients (the "Chow parameters").*

Beyond the Chow characterization, LTFs have bounded noise sensitivity [Harsha et al., 2014], which is important for the multi-layer analysis in Section 2.8.

2.8 Multi-Layer Spectral Composition

A deep binary network computes a composition of the form $f_L \circ \sigma \circ f_{L-1} \circ \sigma \circ \cdots \circ \sigma \circ f_1$, where each f_ℓ is a binary linear map and $\sigma = \mathrm{sgn}$ is the sign activation. Understanding the spectral properties of this composition is essential for our framework.

Composition identity. When f and g are composed as $h(x_1,\ldots,x_{nm}) = g(f(x^{(1)}),\ldots,f(x^{(m)}))$ with disjoint blocks, a standard identity (cf. O'Donnell [2014, Chapter 9]) gives

$$\mathrm{Stab}_\rho[h] = \sum_{T\subseteq[m]} \widehat{g}(T)^2 \big(\mathrm{Stab}_\rho[f]\big)^{|T|} = \mathrm{Stab}_{\mathrm{Stab}_\rho[f]}[g].$$

Whether this implies contraction (i.e., decreasing stability with depth) depends on additional assumptions about balance and non-degeneracy of the inner functions. We treat the depth-wise evolution of noise stability as an empirical question studied in Section 6, rather than asserting a blanket contraction inequality.

Implications for trainability. If composing layers causes noise stability to decay, this is the spectral manifestation of the gradient vanishing problem. Functions that maintain high noise stability across compositions—those with strong low-degree spectral concentration—are the functions most likely to remain trainable in deep binary networks.

2.9 Group-Theoretic Perspective

The Boolean hypercube $\Omega_n = \{-1, +1\}^n$ is isomorphic to the group $\mathbb{Z}_2^n$ under coordinate-wise multiplication. The characters of this group are exactly the parity functions χ_S, and the Fourier expansion on Ω_n is a special case of Fourier analysis on finite abelian groups.

Classical ($\mathbb{R}/\mathbb{Z}$)	**Boolean ($\mathbb{Z}_2^n$)**	**Role**
$e^{2\pi ikx}$	$\chi_S(x) = \prod_{i \in S} x_i$	Basis functions
DFT / FFT	WHT / FWHT	Fast transform
$\sum \lvert\hat{f}(k)\rvert^2 = \lVert f\rVert_2^2$	$\sum \widehat{f}(S)^2 = \lVert f\rVert_2^2$	Parseval
$f \hat{*} g = \hat{f} \cdot \hat{g}$	$\widehat{f * g} = \hat{f} \cdot \hat{g}$	Convolution thm.
Complex-valued	± 1-valued	Basis alphabet

Connection to Reed–Muller codes. There is a deep link between the Walsh–Hadamard basis and Reed–Muller codes. The r-th order Reed–Muller code $\mathrm{RM}(r, n)$ consists of the evaluation vectors of all multilinear polynomials of degree $\leq r$ over $\mathrm{GF}(2)$, which correspond exactly to the ± 1-valued functions whose Fourier support lies in $\{S : |S| \leq r\}$. Our degree-k spectral parameterization optimizes within this subspace before the sign projection. However, the sign of a degree-k polynomial is *not* generally a degree-k polynomial—$\mathrm{sgn}(f^{\leq k})$ can have Fourier support up to degree n—so the output of the sign projection does not lie in $\mathrm{RM}(k, n)$.

The connection is nonetheless valuable in three ways:

(i) **Minimum distance.** $\mathrm{RM}(r, n)$ has minimum distance 2^{n-r}. For our setting with $r = k$ and $n = \log_2 d$, this means any two distinct degree-$\leq k$ codewords differ in at least $d/2^k$ positions. This provides a separation guarantee: if the spectral training converges to two different coefficient vectors, the corresponding binarized weights will differ substantially. At $k = 3$ and $d = 1024$, the minimum distance is 128 positions.

(ii) **Decoding perspective.** The sign projection $\mathrm{sgn}(f^{\leq k})$ can be viewed as a form of "threshold decoding" from the continuous relaxation space to $\{-1, +1\}^d$. Classical Reed–Muller decoding algorithms (e.g., Reed's majority-logic decoder) provide alternative projection strategies that are guaranteed to recover the correct codeword when the noise level is below 2^{n-r-1}. Exploring whether such decoders outperform the simple sign projection is a direction for future work.

(iii) **List decoding.** When the truncated approximation $f^{\leq k}$ is far from any single codeword, list-decoding results for Reed–Muller codes guarantee the existence of a small list of nearby codewords. This could inform a beam-search variant of the sign projection that explores multiple binarizations.

3 Theoretical Contributions

This section presents our new theoretical results. These range from formal theorems with complete proofs to precisely stated conjectures with supporting evidence. We distinguish clearly between the two.

3.1 Approximation by Degree-Truncated Spectral Parameterization

Our spectral parameterization represents each row of a weight matrix using only Fourier coefficients up to degree k. The following result, which applies standard Boolean Fourier techniques to the truncation setting, quantifies the error introduced.

Proposition 3.1 (Degree-k Truncation Misclassification Bound)**.** *Let $f\colon \{-1,+1\}^n \to \{-1,+1\}$ be a Boolean function with total influence $\mathbf{I}[f]$. Define the degree-k truncation $f^{\leq k}(x) = \sum_{|S|\leq k} \widehat{f}(S)\,\chi_S(x)$, and let $\tilde{f} = \operatorname{sgn}(f^{\leq k})$.*[1] *Then:*

$$\mathbb{P}_x[\tilde{f}(x) \neq f(x)] \leq \sum_{|S|>k} \widehat{f}(S)^2. \tag{15}$$

If additionally $\frac{e\,\mathbf{I}[f]}{k+1} < 1$, then by the level-$k$ inequality (Equation (13)) summed over degrees $> k$:

$$\sum_{|S|>k} \widehat{f}(S)^2 \leq \frac{\left(\frac{e\,\mathbf{I}[f]}{k+1}\right)^{k+1}}{1-\frac{e\,\mathbf{I}[f]}{k+1}}. \tag{16}$$

Proof. For (15): $\tilde{f}(x) \neq f(x)$ implies $f(x)\cdot f^{\leq k}(x) \leq 0$. Since $f(x)^2 = 1$, we have $f(x)\cdot f^{\leq k}(x) = 1 - f(x)\cdot f^{>k}(x)$, where $f^{>k} = f - f^{\leq k}$. By Markov's inequality and Parseval:

$$\mathbb{P}[\tilde{f} \neq f] \leq \mathbb{P}[|f^{>k}(x)| \geq 1] \leq \mathbb{E}[(f^{>k})^2] = \sum_{|S|>k} \widehat{f}(S)^2.$$

For (16), apply the level-j inequality for each $j > k$ and sum:

$$\sum_{|S|>k} \widehat{f}(S)^2 = \sum_{j=k+1}^{n} \sum_{|S|=j} \widehat{f}(S)^2 \leq \sum_{j=k+1}^{n} \left(\frac{e\cdot \mathbf{I}[f]}{j}\right)^j \leq \sum_{j=k+1}^{\infty} \left(\frac{e\cdot \mathbf{I}[f]}{k+1}\right)^j.$$

When $r := \frac{e\,\mathbf{I}[f]}{k+1} < 1$, this geometric series sums to $\frac{r^{k+1}}{1-r}$. □

Remark 3.2 (Vacuity in the target regime). We stress that the *first* bound (15)—the Markov/Parseval tail $\mathbb{P}[\tilde{f} \neq f] \leq \sum_{|S|>k} \widehat{f}(S)^2$—is always valid and directly useful: it says the misclassification rate is exactly the spectral energy above degree k, which is

[1] We adopt the convention $\operatorname{sgn}(0) = +1$; for Lebesgue-typical continuous coefficient vectors, no input maps to exactly zero.

the quantity we measure empirically. What is vacuous in our regime is the *geometric enhancement* (16), which attempts to bound that tail energy using the total influence.

In our setting, each row w_i is a Boolean function on $n = \log_2 d$ variables (the column-index bits), so its total influence satisfies $\mathbf{I}[f] = \sum_{i=1}^{n} \mathrm{Inf}_i[f] \leq n$ (since each variable's influence is at most 1). For a "typical" Boolean function on n bits—one whose spectral mass follows the binomial distribution $\mathrm{Bin}(n, 1/2)$—the expected total influence is $\mathbb{E}[\mathbf{I}[f]] = n/2$. At $n = 10$, this gives $\mathbf{I}[f] \approx 5$ and $e \cdot 5 \approx 13.6$; the geometric bound would require $k + 1 > 13.6$, i.e., $k \geq 14$, which already exceeds the maximum degree $n = 10$. Even in the most favorable case of a dictatorship (a function depending on a single variable, $\mathbf{I}[f] = 1$), the bound requires $k \geq 2$ just to activate, providing no useful guarantee at our target truncation degree of $k = 3$.

We explicitly acknowledge that the level-k inequality machinery provides essentially no theoretical guarantee in the small-n regime of interest. This is not a deficiency of our framework but a known limitation of these combinatorial tools at small scale. The limitation fundamentally motivates our empirical pilot study (Section 5): if spectral concentration holds at $n = 10$, it must be driven by learned structure beyond what worst-case influence bounds capture.

Note on two different influence quantities. The total influence discussed here ($\mathbf{I}[f] \leq n = 10$) is for the row w_i viewed as a Boolean function on $n = \log_2 d$ column-index bits. This is distinct from the LTF influence $\mathbf{I}[\mathrm{sgn}(w^\top x)] \approx \sqrt{2d/\pi} \approx 25.5$ computed in Section 4.1, which treats $\mathrm{sgn}(w^\top x)$ as a function of $d = 1024$ activation variables. These are different functions on different domains; the former governs the spectral truncation bound here, while the latter governs the information bottleneck analysis.

3.2 Normalization Propagation in Hadamard Networks

A central concern (Section 1.2) is managing activation variance without learned scaling factors. In BitNet, the learned α per layer serves exactly this role: it rescales the output of a binary linear layer (whose raw magnitude is $O(\sqrt{d})$) to match the expected input scale of the next layer. We show that for Hadamard-structured layers, the required scaling is a fixed, analytically known constant.

Proposition 3.3 (Normalization through Hadamard layers)**.** *Consider a single Hadamard-structured layer:* $y = \frac{1}{\sqrt{d}} H_n \cdot \mathrm{diag}(s) \cdot x$ *where* $s \in \{-1, +1\}^d$ *and* $x \in \mathbb{R}^d$*. Then:*

(i) $\|y\|_2 = \|x\|_2$ *(isometry).*

(ii) If $x \in \{-1, +1\}^d$*, each entry* y_j *is an integer in* $\{-d, -d+2, \ldots, d-2, d\}$ *divided by* $\sqrt{d}$*, hence* $|y_j| \leq \sqrt{d}$*.*

(iii) For x *uniformly random in* $\{-1, +1\}^d$*, by the central limit theorem as* $d \to \infty$*, each* y_j *converges in distribution to* $\mathcal{N}(0, 1)$*.*

Proof. (i) follows from the orthogonality of H_n and the fact that $\mathrm{diag}(s)$ preserves norms. (ii) follows because $H_n \cdot (s \odot x)$ has integer entries (being a sum of d terms each ± 1). (iii) follows from the Berry–Esséen theorem applied to the sum $y_j = d^{-1/2} \sum_i H_{n,ji} s_i x_i$, where the summands are independent ± 1 random variables. □

Remark 3.4 (Multi-layer normalization). For a sequence of L Hadamard-structured layers with sign activations between them, each layer divides by the fixed constant $\sqrt{d}$ (as in Proposition 3.3). These divisors are known at initialization; in implementation they can be handled via precomputed fixed-point scaling without learned parameters or runtime floating-point arithmetic.

However, after the sign activation, the output is back in $\{-1, +1\}^d$ with $\|y\|_2^2 = d$, preventing accumulation. The signal flow through one complete layer (Hadamard multiply $\to$ sign activate) maps $\{-1, +1\}^d \to \{-1, +1\}^d$, and the magnitude of the pre-activation values is $O(\sqrt{d})$ with high probability.

Remark 3.5 (On the accumulation concern). The reviewer raised the concern that normalization factors might accumulate exponentially across layers. Remark 3.4 notes this does not happen: the discrete nature of the sign activation prevents unbounded accumulation. This is a genuine structural advantage of the binary activation regime over networks with unbounded activations. In the regime where activations are real-valued (our easier experimental setting, Section 4.1 option (a)), accumulation is a real concern, and we propose handling it by inserting an explicit $1/\sqrt{d}$ normalization (a fixed constant, not learned) after each layer.

Remark 3.6 (Normalization for general binary matrices). The CLT result (Proposition 3.3(iii)) extends to *any* fixed binary weight matrix $W \in \{-1, +1\}^{d \times d}$, not only Hadamard-structured ones. For uniform $x \in \{-1, +1\}^d$, each entry of $d^{-1/2} W x$ is

$$(d^{-1/2} W x)_j = d^{-1/2} \sum_{i=1}^{d} w_{ji}\, x_i,$$

a normalized sum of d independent ± 1 random variables (since $w_{ji} x_i \in \{-1, +1\}$ when $w_{ji} \in \{-1, +1\}$ and x_i are iid uniform $\{-1, +1\}$). By Berry–Esséen, this converges to $\mathcal{N}(0, 1)$ as $d \to \infty$, regardless of the specific row $w_j \in \{-1, +1\}^d$. Moreover, the variance-preservation identity

$$\mathbb{E}\big[\|d^{-1/2}\, W x\|_2^2\big] = d^{-1} \sum_{j=1}^{d} \sum_{i=1}^{d} w_{ji}^2\, \mathbb{E}[x_i^2] = d^{-1} \cdot d \cdot d = d = \mathbb{E}\big[\|x\|_2^2\big]$$

holds for every $W \in \{-1, +1\}^{d \times d}$, since $w_{ji}^2 = 1$.

The distinction from the Hadamard case is that property (i) of Proposition 3.3—the *isometry* $\|y\|_2 = \|x\|_2$—is specific to orthogonal matrices. For a generic $W \in$

$\{-1,+1\}^{d\times d}$, $W^\top W \neq d\,I$, so $\|Wx\|_2$ depends on the alignment of x with the singular vectors of W and may deviate from $\sqrt{d}\,\|x\|_2$ for individual inputs, even though it equals $\sqrt{d}\,\|x\|_2$ in expectation over uniform binary x.

In summary, the $1/\sqrt{d}$ normalization is a variance-preserving scaling for *all* binary weight matrices, but an exact isometry only for (scaled) orthogonal ones such as Hadamard.

3.3 Formal Conjectures

We state two conjectures that guide the experimental program and which we aim to resolve (or refine) during the thesis.

Conjecture 3.7 (Spectral concentration of trained binary weights)**.** *Let $W \in \{-1,+1\}^{d\times d}$ be a weight matrix from a binary neural network trained to non-trivial performance on a natural language modeling task. For each row w_i of W, viewed as a Boolean function on $n = \log_2 d$ variables via the binary column-index encoding, the effective spectral degree k_i^* (the minimum degree k such that $\sum_{|S|\le k} \widehat{w}_i(S)^2 \ge 0.95$) satisfies $k_i^* \ll n/2$ with high probability over rows. Specifically, at least 90% of rows satisfy*

$$k_i^* \le 3 \quad (\textit{for } n = 10). \tag{17}$$

By comparison, a random Boolean function has its 95%-energy threshold at $k^* \approx n/2 + O(\sqrt{n})$.

Evidence. (i) LTFs, which are the building blocks of binary networks, have strong low-degree spectral characterizations (e.g., unique determination by Chow parameters) and bounded noise sensitivity [Harsha et al., 2014]. (ii) The implicit bias literature suggests gradient-trained networks converge toward "simple" functions in the sense of low norm or low rank [Neyshabur et al., 2015]. We note explicitly that this notion of simplicity—defined in continuous parameter space—does not directly imply low Fourier degree in the Boolean sense. However, there is an indirect connection: for an LTF $\mathrm{sgn}(w^\top x)$ with $w \in \mathbb{R}^d$, Berry–Esséen arguments give $\mathbf{I}[\mathrm{sgn}(w^\top x)] \approx \sqrt{2/\pi} \cdot \|w\|_1/\|w\|_2$, so low-norm weight vectors yield bounded-influence LTFs, which in turn have bounded spectral tails by the level-k inequality (Equation (13)). Whether this chain of implications holds for trained networks is precisely the empirical question our pilot study addresses. (iii) Random Boolean functions have flat spectra, so any concentration observed in trained weights is evidence of learned structure. We test this conjecture directly in Section 5.

Conjecture 3.8 (Spectral parameterization improves optimization)**.** *For training binary neural networks on language modeling tasks, spectral parameterization at degree $k = O(\log d)$ achieves lower final loss than weight-domain STE with the same training budget, provided the degree penalty (Section 4.7) is used.*

Gradient conditioning analysis. We state explicitly a fact that constrains the conjecture: the Walsh–Hadamard basis matrix Φ is orthogonal (up to the known scaling $2^{-n/2}$). Consequently, the Hessian of the loss with respect to spectral coefficients $\widehat{w}$ is related to the Hessian with respect to spatial weights w by

$$\nabla^2_{\widehat{w}}\mathcal{L} = \Phi\, \nabla^2_w \mathcal{L}\, \Phi^\top,$$

which is a similarity transform that *preserves the condition number*: $\kappa(\nabla^2_{\widehat{w}}\mathcal{L}) = \kappa(\nabla^2_w\mathcal{L})$. Without further structure, the spectral and weight-domain landscapes are isometric and no optimization advantage should be expected from the change of basis alone.

The advantage, if any, must therefore arise entirely from two non-trivial additions:

(i) *Degree truncation* restricts the parameterization to $\binom{n}{\leq k}$ coefficients, projecting the optimization onto a $\binom{n}{\leq k}$-dimensional subspace. The effective Hessian becomes the $\binom{n}{\leq k} \times \binom{n}{\leq k}$ principal submatrix of the full spectral Hessian, which generically has a *better* condition number than the full $2^n \times 2^n$ Hessian. The intuition is that high-degree spectral directions, which correspond to fine-grained combinatorial structure, tend to have small curvature in the loss landscape—they are the flat, noisy directions that slow convergence. We do not assert this universally; whether it holds for the landscapes encountered in binary language model training is an empirical question tested directly in ablation A1 (varying k). At $k = 3$ and $d = 1024$, this reduces the optimization from 1024 to 176 dimensions per row, a $\approx 6\times$ compression that concentrates gradient signal on the structurally important subspace.

(ii) The *degree penalty* (Section 4.7) adds curvature $\lambda|S|^p$ to each spectral direction, lifting the small eigenvalues of the Hessian along high-degree coordinates. This explicitly improves the condition number without changing the search space dimension.

The conjecture is that the combination of a smooth continuous landscape (Fourier coefficients) with these sparsity-inducing structures yields better optimization than the discontinuous weight-domain STE, where no analogous low-complexity subspace is available. We test this in ablation A1 (varying k) and A4 (varying regularization strength).

4 Architecture and Methods

The experimental plan assumes the pilot study (Section 5) is completed first; its outcomes determine whether the spectral parameterization or Hadamard-structured layers are prioritized.

4.1 Target Architecture: Proof-of-Concept Scope

We consider a decoder-only Transformer with embedding dimension $d \in \{256, 512, 1024\}$ (powers of 2 for efficient WHT), $h \in \{4, 8\}$ attention heads, $L \in \{4, 8, 12\}$ layers, and context length $T \in \{512, 1024\}$ tokens. We call this the *Spectral Binary Transformer* (SBT).

Binary constraint. We distinguish two senses of "binary" in this proposal, corresponding to the two weight construction approaches developed below:

(a) **Elementwise binary weights** (spectral approach, Modules 1–2): each entry of the deployed weight matrix is in $\{-1, +1\}$, obtained by sign-projecting the spectrally synthesized continuous values. The matrix-vector product Wx for $W \in \{-1, +1\}^{d \times d}$ reduces to additions and subtractions.

(b) **Binary-factored structured weights** (Hadamard approach, Section 4.4): the weight matrix is expressed as a product of Hadamard–sign–permutation stages (Equation (24)), whose learnable components—sign vectors $s_\ell \in \{-1, +1\}^d$ and permutation matrices P_ℓ—are discrete. The effective dense matrix is *not* elementwise ± 1 (see Section 4.4); however, the factored form is never materialized at inference. Each stage is applied sequentially using $O(d \log d)$ additions, sign flips, and index permutations, with fixed $d^{-1/2}$ normalization per stage.

Both approaches eliminate BitNet's learned per-layer floating-point scaling factor α. For Hadamard-structured layers, the replacement $1/\sqrt{d}$ is derived as an exact isometry (Proposition 3.3); for general binary weight matrices (the spectral approach), the same $1/\sqrt{d}$ scaling preserves activation variance in expectation, though without the isometry guarantee (Remark 3.6). During training, both approaches use continuous auxiliary parameters (spectral coefficients or Sinkhorn relaxations) that are discretized or discarded for inference. Unless otherwise stated, claims in this proposal apply to regime (a); where the Hadamard approach differs, we note this explicitly.

Residual connections. In a standard Transformer, residual connections ($x_{\ell+1} = x_\ell + \mathrm{SubLayer}(x_\ell)$) are essential for training beyond a few layers. However, if both the sublayer output and the residual are binary, their sum takes values in $\{-2, 0, +2\}^d$, breaking

the $\{-1, +1\}$ invariant. Applying sgn to re-binarize discards the ternary structure (in particular, $\mathrm{sgn}(0)$ is undefined or arbitrary), losing information rather than preserving it.

We resolve this by maintaining the main residual stream in higher precision (INT8 or FP16). Concretely, the signal flow through one Transformer sub-layer is:

$$x_{\ell+1} = x_\ell + W_\ell \cdot \mathrm{sgn}(x_\ell),$$

where $x_\ell \in \mathbb{R}^d$ is the (higher-precision) residual, $\mathrm{sgn}(x_\ell) \in \{-1, +1\}^d$ is the binarized input to the weight matrix, $W_\ell \in \{-1, +1\}^{d\times d}$ is the binary weight matrix, and the matrix-vector product $W_\ell \cdot \mathrm{sgn}(x_\ell)$ is computed via additions and subtractions only ($O(d^2)$ integer operations). The result is an integer vector in $[-d, +d]^d$ which is added back to the higher-precision residual. This follows the standard practice in 1-bit LLMs: BitNet [Wang et al., 2023] similarly maintains 8-bit activations and applies quantization only at the inputs to each linear layer.

This design means that in Regime (b) (binary activations), "binary" refers specifically to the inputs of the weight matrices, not to the residual stream itself. The computational savings come from the $O(d^2)$ binary matmul, which dominates the cost; the $O(d)$ higher-precision residual addition is negligible by comparison.

Activation regimes. We explore two regimes:

(a) **Real activations** (easier): activations are real-valued, with $1/\sqrt{d}$ fixed normalization after each binary linear layer. This isolates the effect of binary weights.

(b) **Binary activations** (harder): activations are binarized via sgn at each layer. This yields the cleanest binary inference pipeline but introduces the multi-layer composition challenges of Section 2.8. It also creates a severe *information bottleneck*: the sign function discards all magnitude information, compressing each pre-activation from $O(\log d)$ bits down to 1 bit per element. Whether complex syntactic dependencies can survive repeated passage through this bottleneck without continuous scaling is an open question that we monitor via the layer-wise spectral diagnostics and test directly in ablation A5. We treat Regime (b) as a secondary target; if binary activations everywhere lead to training instability or severe perplexity degradation (e.g., gradient collapse or perplexity explosion), we retain Regime (a) as the primary configuration.

Mutual information analysis of the sign bottleneck. We quantify the information constraint imposed by binary activations. For a single layer with input $x \in \{-1, +1\}^d$ (uniform) and fixed binary weight matrix $W \in \{-1, +1\}^{d\times d}$, the output $y = \mathrm{sgn}(Wx) \in$

$\{-1,+1\}^d$ is deterministic given x, so the mutual information is

$$I(x;\, \mathrm{sgn}(Wx)) = H(\mathrm{sgn}(Wx)) \le d \text{ bits.} \tag{18}$$

By contrast, an FP16 layer's output occupies a representation space of $(2^{16})^d$ distinguishable states—a gap of $16d$ versus d in the log-cardinality of the output space. We stress that this is a *representation capacity* comparison, not a mutual information bound: for a deterministic layer with discrete input, the actual MI satisfies $I(x;\, f(x)) = H(f(x)) \le H(x)$, which is at most d bits for binary input regardless of output precision. The representation gap becomes operationally significant across multiple layers, where the per-layer output entropy constrains the data processing inequality chain (see below).

This bound is not necessarily tight for a specific W: if rows of W induce correlated output bits, $H(\mathrm{sgn}(Wx))$ may be significantly less than d. For generic W, however, each output bit $\mathrm{sgn}(w_i^\top x)$ is balanced (by symmetry of the uniform distribution on $\{-1,+1\}^d$) and distinct rows produce approximately independent outputs (since random binary rows are nearly orthogonal for large d), yielding $H(\mathrm{sgn}(Wx)) \approx d$.

The comparison to task requirements is instructive. Next-token prediction over a vocabulary of size V requires at most $\log_2 V \approx 16$ bits per position. At $d = 1024$, the binary layer provides $\approx 64\times$ this minimum, suggesting adequate per-layer capacity for a single prediction. The binding constraint is *multi-layer information decay*: by the data processing inequality,

$$I(x_0;\, y_L) \le I(x_0;\, y_\ell) \le d \quad \text{for all } 0 \le \ell \le L, \tag{19}$$

where x_0 is the input embedding and $y_\ell = \mathrm{sgn}(W_\ell y_{\ell-1})$ is the activation at layer ℓ. Information about the input can only decrease with depth, and each sign activation is an irreversible compression step. In a real-valued (FP16) network, each intermediate representation admits up to $16d$ bits of entropy (the log-cardinality of the FP16 output space per layer), providing substantially more headroom for preserving input information across layer compositions. While this upper bound is rarely saturated in practice, the qualitative gap—a d-bit entropy ceiling for binary activations versus a $16d$-bit ceiling for FP16—underscores the severity of the sign bottleneck in deep binary networks.

The Boolean Fourier perspective provides additional insight. Each output bit $y_i = \mathrm{sgn}(w_i^\top x)$ is a balanced LTF whose total influence is, by Berry–Esséen,

$$\mathbf{I}[\mathrm{sgn}(w^\top x)] \;\approx\; \sqrt{2/\pi} \;\cdot\; \frac{\|w\|_1}{\|w\|_2}. \tag{20}$$

For binary $w \in \{-1,+1\}^d$, $\|w\|_1 = d$ and $\|w\|_2 = \sqrt{d}$, giving $\mathbf{I}[f] \approx \sqrt{2d/\pi}$. At $d = 1024$ this yields $\mathbf{I}[f] \approx 25.5$: despite nominally depending on all d inputs, each output bit has

an effective sensitivity scaling as $O(\sqrt{d})$. Low-influence functions transmit information selectively, preserving low-degree structure and discarding high-frequency detail—exactly the filtering behavior predicted by the spectral concentration hypothesis (Conjecture 3.7).

We monitor two diagnostics during training: (i) the empirical output entropy $H(\mathrm{sgn}(W_\ell y_{\ell-1}))$, estimated over minibatches, and (ii) the per-row total influence $\mathbf{I}[f_i]$, computed from spectral coefficients. If $H(\mathrm{sgn}(Wx))$ at any layer drops below $\log_2 V$, the bottleneck is binding and regime (b) is information-limited for the given model dimension d.

Input encoding. We consider learned embeddings (initially full precision) and fixed *Hadamard embeddings* in which each token is assigned a row of a Hadamard matrix.

Embedding and output projection. The token embedding table is a $V \times d$ matrix, where V is the vocabulary size (typically $V \approx 50{,}000$). For a standard Transformer at $d = 1024$, this amounts to $\approx$50M parameters—a significant fraction of a small model's total parameter budget. Similarly, the output projection from hidden dimension to vocabulary logits is a $d \times V$ matrix (often tied with the embedding). These are substantial parameter stores that must be explicitly addressed in a "purely binary" model. We explore three strategies in ablation A6 (Section 6.4):

(i) **Learned binary embeddings.** Each token's embedding is a vector in $\{-1, +1\}^d$, trained via STE or spectral parameterization. This forces tokens to be represented as vertices of the hypercube, severely limiting the capacity for fine-grained semantic similarity.

(ii) **Fixed Hadamard embeddings.** Assign each token a row of a Hadamard matrix (or a block-diagonal Hadamard matrix when $V > d$). This provides orthogonal, analytically normalized embeddings with no learned parameters, but sacrifices the ability to learn task-specific token representations.

(iii) **Full-precision embeddings (pragmatic baseline).** Keep the embedding in FP16 and apply binarization only to the internal layers. This is the approach taken by BitNet and allows fair comparison. The embedding is a one-time lookup per token (no arithmetic), so its precision has minimal impact on inference throughput.

For the output projection, we test both weight tying (sharing the embedding matrix) and an independent binary projection. Note that the output layer produces logits for softmax, which require continuous values—the output projection is therefore the layer where binary weights are most constraining, since the logits must resolve fine-grained probability differences across V tokens.

Inference-time precision audit. Because Section 1.2 specifically critiques existing methods for "precision leakage," intellectual honesty demands a complete accounting of where non-binary arithmetic enters our own inference pipeline. We enumerate every component:

- **Core weights:** Strictly 1-bit (± 1). The matrix-vector product Wx for $W \in \{-1, +1\}^{d \times d}$, $x \in \{-1, +1\}^d$ requires only additions and subtractions (implementable via XNOR + popcount on hardware that supports it), producing integer outputs in $[-d, +d]$.

- **Scaling factors:** The fixed constant $1/\sqrt{d}$ replaces BitNet's learned per-layer α. For power-of-two d, this is a right bit-shift (e.g., $1/\sqrt{1024} = 1/32 = 2^{-5}$); for general d, it is a precomputed fixed-point multiplier. No learned floating-point parameters.

- **Bias terms:** We use no additive bias in any linear layer, following the convention of recent binary and low-bit models. This eliminates one potential source of full-precision parameters.

- **Residual stream:** Maintained in INT8 or FP16 to preserve signal fidelity across layers, as described in the "Residual connections" paragraph above. This is the most significant non-binary component by volume (one d-dimensional vector per position per layer).

- **Embeddings:** In the pragmatic baseline (option (iii) above), the token embedding lookup is FP16. This is a table lookup, not an arithmetic operation, so it does not affect inference throughput. In the binary embedding variants (options (i)–(ii)), the embeddings are in $\{-1, +1\}^d$.

- **Positional encodings:** Full-precision arithmetic. Absolute (sinusoidal or learned) encodings are added to the embedding vectors before the first layer. Rotary positional encodings (RoPE) are applied to the projected query and key vectors *after* the binary linear projection, requiring element-wise full-precision rotation.

- **Normalization:** RMSNorm computes a full-precision root-mean-square statistic and applies a learned gain parameter (one scalar per feature dimension). LayerNorm additionally computes the mean and applies a learned bias. Both require full-precision arithmetic, but the parameter count is small: $O(d)$ per layer, compared to $O(d^2)$ for the weight matrices.

- **Attention softmax:** In Option A (our recommended starting point), the standard softmax operates at full precision over the $T \times T$ attention score matrix. This is a per-head, per-layer operation involving $O(T^2)$ floating-point exponentials and a normalization division—non-trivial but confined to the attention scores, not to the

$O(d^2)$ weight matrices. Options B and C (Section 4.5) progressively reduce this footprint.

In summary, the SBT eliminates BitNet's *learned* scaling factors entirely and confines non-binary arithmetic to components that are either structurally unavoidable (normalization, softmax), low-volume ($O(d)$ residual additions vs. $O(d^2)$ matmuls), or shared with BitNet (embeddings, positional encodings). The core $O(d^2)$ matrix-vector multiplications—which dominate both computation and memory bandwidth—are strictly binary.

Non-power-of-two dimensions. The power-of-two constraint ($d = 2^n$) is natural for the Walsh–Hadamard framework but limits practical applicability: many standard model dimensions ($d = 768, 2048, 3072$) are not powers of two, though they factor as products of a power of two and a small odd factor (e.g., $768 = 3 \times 256$, $3072 = 3 \times 1024$). We accommodate such dimensions via *block-diagonal decomposition*: partition each weight matrix into blocks of size $2^k \times 2^k$ and apply the spectral parameterization independently within each block. For $d = 768 = 3 \times 256$, this gives 3 blocks of 256×256, each with its own Walsh–Fourier representation. The block structure introduces a prior that inter-block interactions are axis-aligned; a learnable pre-block permutation can mitigate this constraint. Alternatively, rows can be zero-padded to the next power of two, at the cost of wasted dimensions. For proof-of-concept experiments, we restrict to power-of-two d; for scaling experiments, we implement the block-diagonal approach.

4.2 Module 1: Spectral Layer Parameterization

4.2.1 Row-Wise Spectral Decomposition

Each row w_i of a weight matrix is treated as a Boolean function $w_i\colon \{-1,+1\}^n \to \{-1,+1\}$, where $n = \log_2(d_{\text{row}})$ is determined by the row length. For square attention projection matrices ($W_Q, W_K, W_V, W_O \in \{-1,+1\}^{d\times d}$), $n = \log_2 d$. For the rectangular feed-forward matrices, the up-projection $W_{\text{up}} \in \{-1,+1\}^{d\times 4d}$ has rows of length $4d$, giving $n = \log_2(4d) = \log_2(d) + 2$; the down-projection $W_{\text{down}} \in \{-1,+1\}^{4d\times d}$ has rows of length d, so $n = \log_2(d)$, same as the attention matrices. The column index $j \in \{0, \ldots, d_{\text{row}} - 1\}$ is mapped to a hypercube vertex $\text{bin}(j) \in \{-1,+1\}^n$ via its binary representation. Each row has a Walsh–Fourier expansion:

$$w_i(j) = \sum_{S\subseteq[n]} \widehat{w}_i(S)\, \chi_S(\text{bin}(j)). \tag{21}$$

We parameterize only coefficients up to degree k, giving $\binom{n}{\leq k}$ parameters per row. The following table shows how this parameter count compares to the full row dimension d:

d	$n = \log_2 d$	$\binom{n}{\leq 2}$	$\binom{n}{\leq 3}$	$\binom{n}{\leq 4}$	d (full)
256	8	37	93	163	256
512	9	46	130	256	512
1024	10	56	176	386	1024

At degree $k = 3$ and $d = 1024$, the spectral parameterization uses 176 free parameters per row—an $\approx 6\times$ compression of the optimization landscape relative to the full $d = 1024$ weight-domain parameterization. For the FFN up-projection rows (length $4d = 4096$, $n = 12$), $\binom{12}{\leq 3} = 299$ parameters per row, still a $\approx 14\times$ compression versus the full 4096. This compression is an advantage if spectral concentration holds (fewer parameters, better-conditioned optimization) but a bottleneck if the target functions genuinely require high-degree terms.

4.2.2 The Column-Indexing Problem

The row-wise decomposition requires choosing how column indices map to hypercube vertices. This choice is not canonical: different binary encodings of the index set $\{0, \ldots, d-1\}$ yield different Walsh–Fourier spectra for the same weight vector.

This is a fundamental issue for the framework. We correctly note below that bit permutations preserve the degree profile, but the more critical question is whether the "natural" binary encoding has any principled relationship to the semantic structure of the learned weights. A weight matrix W transforms activations, where each column index corresponds to an input neuron. The standard binary encoding imposes a hierarchical grouping (e.g., columns 0–127 share the most significant bit) that has no *a priori* connection to the order of neurons.

If we observe spectral concentration under the natural binary encoding, it could be an artifact of that encoding accidentally aligning with the weight structure, or it could reflect a deep inductive bias of the training process. Conversely, an absence of concentration under the natural encoding does not necessarily disprove the utility of spectral methods—it might simply mean the natural encoding is the wrong one for that matrix.

To address this, we propose three strategies:

(i) **Standard binary encoding.** Use the natural binary encoding as a default. Our pilot study (Section 5) explicitly tests against random column permutations to control for encoding artifacts.

(ii) **Gray code encoding.** Gray codes ensure adjacent indices differ in exactly one bit, which may improve spectral locality if adjacent neurons are correlated. We test this as an ablation.

(iii) **Learned encoding.** Include a learnable permutation (via Sinkhorn parameterization) that maps column indices to hypercube vertices. During training, each Sinkhorn iteration is $O(d^2)$ per layer; for large d this cost is non-trivial and we report wall-clock overheads in our ablations. At inference, the permutation is frozen and amounts to a fixed column reordering with no runtime cost.

We take (i) or (ii) as the default for the spectral path; (iii) is an optional ablation (A8). We do not rely on learned permutations for the main results. Hadamard-structured layers (Section 4.4) provide a column-indexing-free alternative without $O(d^2)$ training overhead and are the recommended fallback if encoding sensitivity is prohibitive.

For the Hadamard-structured layers (Section 4.4), the column-indexing issue does not arise, since the Hadamard structure is defined intrinsically rather than via an arbitrary encoding.

Alternating optimization and the encoding circularity. There is a circularity in the column-encoding problem: the optimal encoding depends on the weight function being compressed (since it determines which encoding yields a low-degree spectral profile), but the spectral decomposition requires fixing an encoding first. When using strategy (iii) above, our framework handles this via alternating optimization: fix the encoding, optimize the spectral coefficients; then fix the coefficients, update the encoding via the Sinkhorn permutation gradient. This alternation is implicit in Algorithm 1 (Algorithm 1), where the spectral coefficients and Sinkhorn parameters receive joint gradient updates.

Standard results on block coordinate descent guarantee convergence to a stationary point when each subproblem is convex or has a unique minimizer, but neither condition holds here—the spectral coefficient optimization is non-convex (due to the sign projection) and the permutation optimization is combinatorial. In practice, we expect convergence to a local optimum that depends on initialization. Ablation A8 (Section 6.4) tests sensitivity to the initial encoding by comparing convergence from multiple random initializations. A natural question for future work is whether there exist function classes for which *every* column encoding yields low-degree concentration, rendering the choice benign. Symmetric Boolean functions (which are invariant to input permutations) are one such class, but they are too restrictive to model realistic weight matrices.

4.3 Module 2: Spectral-to-Binary Projection

After each gradient update to the spectral coefficients, we must project back to a valid Boolean function. We propose and compare three strategies.

4.3.1 Hard Sign Projection

Compute the spatial-domain representation via inverse WHT and apply the sign function:

$$w_i(j) \leftarrow \operatorname{sgn}\left(\sum_{\substack{S\subseteq[n]\\|S|\le k}} \widehat{w}_i(S)\,\chi_S(\operatorname{bin}(j))\right). \tag{22}$$

4.3.2 Randomized Rounding

Interpret the continuous spatial value as a probability:

$$w_i(j) = \begin{cases} +1 & \text{with probability } p(z_i(j)),\\ -1 & \text{otherwise,}\end{cases} \tag{23}$$

where $z_i(j) = \sum_{|S|\le k} \widehat{w}_i(S)\,\chi_S(\operatorname{bin}(j))$ and $p(z) = (1+\operatorname{clip}(z,-1,1))/2$ maps the continuous value to a probability. Gradients flow via Gumbel-softmax or REINFORCE.

4.3.3 Spectral Norm Projection

Enforce Parseval: $\widehat{w}_i \leftarrow \widehat{w}_i / \|\widehat{w}_i\|_2$. Combined with hard sign, this gives a two-stage procedure that replaces learned scaling.

4.3.4 Sparse Spectral Layers (Variant)

If the pilot study (Section 5) reveals that trained binary weights concentrate on a small number of Fourier coefficients, a natural variant is to parameterize each row using only the top-K Walsh coefficients (by magnitude) rather than all $\binom{n}{\le k}$ coefficients at degree $\le k$. This reduces the number of free parameters per row from $\binom{n}{\le k}$ to K, which may improve optimization and generalization. Note that storing K coefficient–index pairs requires $K \times (b_{\text{coeff}} + \lceil \log_2 \binom{n}{\le k} \rceil)$ bits per row, where b_{coeff} is the coefficient precision; at $n = 10$ with 32-bit coefficients, this exceeds the 1024 raw binary bits for $K \gtrsim 24$. The benefit is therefore not raw bit-count compression but rather a reduction in the effective dimensionality of the optimization landscape. The top-K set can be selected per-layer (uniform sparsity) or per-row (adaptive sparsity), and can be updated periodically during training. We include this as an experimental variant in ablation A1 (Section 6.4).

4.4 Alternative Approach: Hadamard-Structured Layers

We build weight matrices from Hadamard sub-blocks:

$$W = \frac{1}{d^{L_H/2}} \prod_{\ell=1}^{L_H} \left[H_n \cdot \operatorname{diag}(s_\ell) \cdot P_\ell\right], \tag{24}$$

where H_n is the Hadamard matrix, each $s_\ell \in \{-1, +1\}^d$, each P_ℓ is a permutation, and L_H is the number of stages. Each stage divides by $\sqrt{d}$ (the same fixed normalization as in Proposition 3.3), giving the $d^{-L_H/2}$ prefactor.

Learnable components and binary scope. The sign vectors s_ℓ are natively binary. The permutations P_ℓ are parameterized via Sinkhorn operators during training [Mena et al., 2018]. We acknowledge that Sinkhorn matrices are continuous—this is another form of shadow state during training. At inference, they are rounded to hard permutations. The sensitivity to this rounding is an empirical question we address in ablation A7 (Section 6.4).

Note that the *effective dense matrix* W produced by the product in (24) is *not* elementwise ± 1 in general—the Hadamard multiplication and normalization yield real-valued entries. The binary constraint in the Hadamard approach applies to the *factor parameters* (s_ℓ and the hard permutations P_ℓ), not to the dense matrix they produce. The advantage is that these factors are natively discrete and the normalization is analytic, eliminating learned scaling factors at inference.

Expressive power. Products of Hadamard-sign-permutation stages define a rich structured family with favorable computational properties. The number of distinct matrices realizable grows rapidly with the stage count L_H, and butterfly-style factorizations suggest that $O(\log d)$ stages suffice to approximate a wide class of structured linear maps [Dao et al., 2022]. We treat the precise approximation quality as an empirical question; see Conjecture 4.1 below.

Conjecture 4.1. *For weight matrices arising in practical language models, $O(\log d)$ Hadamard-sign stages suffice to approximate the optimal binary weight matrix to within rounding error comparable to the spectral projection.*

4.5 Module 3: Spectral Attention Mechanism

We propose three attention variants, ranked by our assessment of feasibility. We will implement all three and compare; our **recommended starting point** is Option A.

Option A: Integer attention with fixed scaling (recommended first). The standard attention computation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{25}$$

is used with $Q = W_Q x$, $K = W_K x$, $V = W_V x$ where all W matrices are binary. The product $QK^\top$ is integer-valued in $[-d_k, +d_k]$. The scaling $1/\sqrt{d_k}$ is a fixed constant (not

learned). The softmax requires non-binary arithmetic, but it operates on a small $T \times T$ matrix (not on the weight matrices), so the floating-point computation is confined to a low-dimensional bottleneck.

Why we recommend this first: It changes the least from the standard Transformer and isolates the effect of binary weights from changes to the attention mechanism. The softmax is the only non-binary component and it operates on attention logits, not on model parameters.

Option B: Hadamard attention. Replace dot-product similarity with Walsh–Hadamard spectral correlation. The key observation is that the standard dot-product attention score $q_i^\top k_j$ can be rewritten via Parseval as a spectral inner product:

$$q_i^\top k_j = d_k \sum_{S \subseteq [\log_2 d_k]} \widehat{q_i}(S)\, \widehat{k_j}(S),$$

where $\widehat{q_i}$ and $\widehat{k_j}$ are the normalized Walsh–Fourier coefficients of q_i and k_j respectively. This motivates replacing the full spectral inner product with a *masked* version that attends only to selected spectral components:

$$A_{ij} = \frac{1}{d_k} \sum_{r=0}^{d_k-1} \left(\mathrm{WHT}(q_i) \odot m \odot \mathrm{WHT}(k_j)\right)_r, \tag{26}$$

where $\mathrm{WHT}(\cdot)$ denotes the fast Walsh–Hadamard transform along the feature dimension and $m \in \{-1, +1\}^{d_k}$ is a learnable binary spectral mask. The mask selects or negates each spectral component, acting as a learned frequency filter.

Gradient flow in Hadamard attention. The gradient with respect to the mask is:

$$\frac{\partial A_{ij}}{\partial m_r} = \frac{1}{d_k} \mathrm{WHT}(q_i)_r \cdot \mathrm{WHT}(k_j)_r,$$

which is itself a product of spectral coefficients — fully compatible with the Walsh–Fourier framework. Since $m \in \{-1, +1\}^{d_k}$, the mask gradient is trained via STE (or Gumbel-softmax) analogously to the weight matrices. The full attention computation is $O(T^2 d_k + T d_k \log d_k)$: the FWHT costs $O(d_k \log d_k)$ per token, and the pairwise scoring costs $O(T^2 d_k)$, both comparable to standard attention.

Normalization. The softmax can be replaced with a simpler normalization (e.g., division by the sum of absolute values), implemented via fixed-point integer arithmetic with precomputed lookup or approximate reciprocal. When the queries and keys are binary (from sign-activated layers), $\mathrm{WHT}(q_i)$ and $\mathrm{WHT}(k_j)$ are integer-valued, so the element-

wise products and sums in (26) are also integer-valued. The only non-integer operation is the final normalization.

Roadmap. We implement Option B after validating Option A. The key experiment is ablation A5 (Section 6.4): compare the three attention variants at the best-performing scale, measuring perplexity, training stability, and the fraction of operations that are strictly binary. If Option B matches Option A's perplexity while reducing the floating-point footprint (by eliminating softmax), it becomes the preferred attention mechanism for the fully binary inference pipeline.

Trade-off: More compatible with binary arithmetic, but departs from the standard attention mechanism in ways that may hurt performance.

Option C: Linear attention with binary feature maps. Use the linearized attention of Katharopoulos et al. [2020]: $\text{Attention}(Q, K, V) \approx \phi(Q)(\phi(K)^\top V)$ where ϕ is a feature map. We set $\phi(x) = \text{sgn}(Rx)$ for a fixed random binary matrix $R \in \{-1, +1\}^{m \times d_k}$, which implements a binary locality-sensitive hash (cf. the hashing-based attention of Kitaev et al. [2020] and the random-feature approach of Choromanski et al. [2021]). The core attention computation then involves only additions and sign comparisons, though normalization still requires fixed-point integer division.

Trade-off: Fully binary-compatible, but linear attention is known to underperform softmax attention, and the binary hash adds additional approximation error. We view this as the long-term target for fully binary inference, not the starting point.

4.6 Module 4: Training Algorithm

Algorithm 1 SBT Training

Require: Training corpus $\mathcal{D}$; degree truncation k; temperature schedule $\{\beta_t\}$

Initialize: $\widehat{w}_i(S) \sim \mathcal{N}(0, 1/\binom{n}{\leq k})$ for all rows i, all $|S| \leq k$
Normalize: $\widehat{w}_i \leftarrow \widehat{w}_i / \|\widehat{w}_i\|_2$
for each training step t with batch $\mathcal{B}_t$ **do**
  $W_{\text{cont}} \leftarrow \widehat{w} \cdot \Phi^\top$ ▷ spectral coeffs × Walsh basis → continuous weights
  $W_{\text{bin}} \leftarrow \tanh(\beta_t \cdot W_{\text{cont}})$ ▷ soft binarization
  $\mathcal{L} \leftarrow \text{cross-entropy}(\text{SBT}(W_{\text{bin}}, \mathcal{B}_t))$
  $\mathcal{L}_{\text{reg}} \leftarrow \mathcal{L} + \lambda \sum_{i,S} |S|^p \widehat{w}_i(S)^2 - \mu \sum_{i,S} \rho^{|S|} \widehat{w}_i(S)^2$
  $\widehat{w} \leftarrow \widehat{w} - \eta_t \nabla_{\widehat{w}} \mathcal{L}_{\text{reg}}$
  $\widehat{w}_i \leftarrow \widehat{w}_i / \|\widehat{w}_i\|_2$ ▷ spectral norm projection
end for
Final: Recompute $W_{\text{cont}} \leftarrow \widehat{w} \cdot \Phi^\top$; set $W_{\text{final}} = \text{sgn}(W_{\text{cont}})$ ▷ hard binarization; discard $\widehat{w}$

Training cost overhead. While the inference pipeline is optimized for binary execution, training the SBT carries a per-step computational overhead compared to standard

weight-domain STE, which has essentially zero reconstruction cost (the sign function is $O(d^2)$ per layer, negligible next to the matmul).

In `SpectralLinear`, the overhead arises from reconstructing the dense weight matrix from spectral coefficients: for each of the d rows, the inverse FWHT costs $O(d \log d)$, giving $O(d^2 \log d)$ per weight matrix per training step. The standard matrix multiplication over a batch of B tokens costs $O(d^2 B)$. Crucially, the FWHT reconstruction is performed *once per step* (to produce the current weight snapshot), not once per token. The amortized overhead ratio is therefore $O(\log d/B)$. At $d = 1024$ ($\log_2 d = 10$) and $B = 256$, this is $10/256 \approx 4\%$—a modest tax. In practice, the FWHT is embarrassingly parallel across rows and maps well to GPU SIMD hardware, so the wall-clock overhead is even smaller than the FLOP ratio suggests.

In `HadamardLinear`, learning the Sinkhorn permutations adds $O(d^2)$ operations per iteration per layer (for each row of the doubly-stochastic matrix). This is comparable to a single matmul and does not change the asymptotic training cost.

The key insight is that spectral training trades a small per-step reconstruction cost for a potentially large reduction in the *number of steps to convergence*—this is exactly the "better inductive bias" claim (Conjecture 3.8), which we test directly in the experiments. We report wall-clock training times alongside FLOPs in Section 6.

The algorithm above uses the $\tanh(\beta_t \cdot)$ soft binarization as the default projection. The two alternative projections from Section 4.3—randomized rounding (Section 4.3.2) and spectral norm projection (Section 4.3.3)—are substituted at line 5 in ablation A2 (Section 6.4). In all cases, the temperature β_t starts small (soft, gradient-friendly) and increases to approximate hard sign by the end of training. The final binarization step produces strictly binary weights.

4.7 Module 5: Spectral Regularization

Degree penalty.

$$\mathcal{R}_{\text{degree}} = \lambda \sum_i \sum_S |S|^p \cdot \widehat{w}_i(S)^2. \tag{27}$$

Noise stability reward.

$$\mathcal{R}_{\text{stability}} = -\mu \sum_i \sum_S \rho^{|S|} \cdot \widehat{w}_i(S)^2. \tag{28}$$

Relationship between the two regularizers. Both regularizers encourage low-degree spectral concentration, but with different weighting profiles. The degree penalty weights $\widehat{w}_i(S)^2$ by $|S|^p$ (polynomial in degree), while the stability reward weights by $\rho^{|S|}$ (exponential decay for $\rho < 1$). For ρ close to 1, the stability reward imposes a gentle

exponential taper that primarily distinguishes between very low and moderately low degrees. For ρ close to 0, it aggressively suppresses all non-constant terms. The degree penalty with $p = 1$ imposes a linear tax on degree, directly penalizing total influence ($\mathbf{I}[f] = \sum_S |S| \cdot \widehat{w}(S)^2$); with $p = 2$, it penalizes the second moment of the spectral distribution over degrees.

Redundancy and Hyperparameter Search. Using both regularizers simultaneously is mathematically coherent but practically redundant. The combined regularizer $\lambda|S|^p - \mu\rho^{|S|}$ is monotonically increasing in $|S|$ for typical hyperparameter values, so both terms pull in the same direction. The key distinction is interpretive: the degree penalty directly controls total influence (a quantity with clear implications for gradient flow and noise sensitivity), while the stability reward directly targets the noise stability functional $\mathrm{Stab}_\rho[f]$, which governs robustness to bit flips in the input.

Introducing these regularizers adds four new hyperparameters (λ, μ, p, ρ) to the existing standard set (learning rate, batch size, etc.) and the spectral-specific ones (degree truncation k, temperature schedule β_t). To make the search space tractable, we set $\mu = 0$ (using only the degree penalty with $p = 1$) for the primary experiments. This directly targets the total influence term that bounds approximation error in Proposition 3.1. We explore the full grid only in ablation A4 (Section 6.4) to verify whether the stability reward offers any unique benefits not captured by the simple degree penalty.

4.8 A Worked Toy Example: Learning the Majority Function

To make the spectral training procedure concrete and to verify the implementation before scaling, we work through a complete example.

Setup. Consider a single binary linear layer $w \in \{-1, +1\}^8$ (so $n = 3$, $d = 8$) followed by a sign activation. The target function is $\mathrm{Maj}_3(x_1, x_2, x_3) = \mathrm{sgn}(x_1 + x_2 + x_3)$.

Exact Fourier spectrum. The majority function on $\{-1, +1\}^3$ outputs $+1$ when at least 2 of 3 inputs are $+1$. Its truth table is:

x_1	x_2	x_3	Maj_3
-1	-1	-1	-1
-1	-1	$+1$	-1
-1	$+1$	-1	-1
-1	$+1$	$+1$	$+1$
$+1$	-1	-1	-1
$+1$	-1	$+1$	$+1$
$+1$	$+1$	-1	$+1$
$+1$	$+1$	$+1$	$+1$

The Fourier coefficients are $\widehat{f}(\emptyset) = 0$ (balanced), $\widehat{f}(\{i\}) = 1/2$ for each i, all degree-2 coefficients are zero, and $\widehat{f}(\{1,2,3\}) = -1/2$. Hence:

$$\mathrm{Maj}_3(x) = \tfrac{1}{2}(x_1 + x_2 + x_3) - \tfrac{1}{2}x_1x_2x_3. \tag{29}$$

Parseval check: $3 \cdot (1/2)^2 + (1/2)^2 = 3/4 + 1/4 = 1$. ✓

Spectral training procedure.

1. Initialize spectral coefficients $\widehat{w}(S) \sim \mathcal{N}(0, 1/4)$ for $|S| \leq 2$ (degree truncation $k = 2$, so 7 coefficients: $1 + 3 + 3$).
2. Forward: compute $z(j) = \sum_{|S|\leq 2} \widehat{w}(S)\chi_S(\mathrm{bin}(j))$ for $j = 0, \ldots, 7$ (this is a partial inverse WHT).
3. $w_{\mathrm{bin}}(j) = \mathrm{sgn}(z(j))$.
4. Compute loss: e.g., $\sum_j (\mathrm{Maj}_3(\mathrm{bin}(j)) - z(j))^2$ (or cross-entropy on a downstream task).
5. Backprop through the continuous z, update $\widehat{w}(S)$, project to unit sphere.

At degree $k = 2$, the degree-3 coefficient $\widehat{f}(\{1,2,3\})$ is not representable, so the spectral training will find the best degree-≤ 2 approximation and then snap to binary via sign. For Maj_3, the degree-1 coefficients carry 3/4 of the spectral energy, so the degree-2 truncation captures the vast majority of the function's structure. The sign projection then recovers the correct Boolean function on all 8 inputs. This exact recovery is somewhat fortunate: it relies on the degree-≤ 1 approximation already having the correct sign on every input. For other Boolean functions (e.g., those with more balanced spectral mass across degrees), the sign projection after low-degree truncation will introduce errors, and the number of bit flips is governed by the tail bound in Proposition 3.1.

This toy example serves as a sanity check for the implementation: the spectral training should converge to coefficients close to the true values, and the spectral diagnostics should show the expected degree profile.

5 Pilot Study: Spectral Concentration in Existing Binary Weights

Before committing to the full experimental program, we propose a targeted pilot study to test the core assumption: *do trained binary weight matrices exhibit spectral concentration?*

5.1 Protocol

Step 1: Obtain binary weight matrices. We use three sources:

(a) **BitNet b1.58 2B4T** (the open-weights 2B model from Microsoft). Extract the ternary $\{-1, 0, +1\}$ weight matrices. For our analysis, we treat these as pseudo-Boolean functions with three values and compute the standard Walsh–Fourier transform.

(b) **Binarized standard Transformer.** Take a trained FP16 Transformer (e.g., GPT-2 small) and binarize each weight matrix via $\mathrm{sgn}(W)$. This gives a (poor) binary model but lets us analyze whether the structure of a trained weight matrix exhibits spectral concentration even after naive binarization.

(c) **STE-trained binary Transformer.** Train a small binary Transformer (our Baseline 2) using standard STE and extract the resulting ± 1 weight matrices.

Non-power-of-two dimensions. The row-wise Walsh–Fourier analysis requires row length $d = 2^n$. When the model dimension is not a power of two (e.g., $d = 768$ for GPT-2 small, $d = 2560$ for BitNet b1.58 2B4T), we run two complementary analyses: (i) blockwise analysis on maximal power-of-two subblocks of each row, and (ii) zero-padding each row to the next power of two. We report both and quantify sensitivity to the choice.

Step 2: Compute Walsh–Fourier spectra. For each weight matrix $W \in \{-1, +1\}^{d \times d}$ (or its padded/blocked version):

1. For each row w_i (length $d = 2^n$), compute the full Walsh–Fourier transform $\widehat{w}_i(S)$ for all $S \subseteq [n]$. This costs $O(d \log d)$ per row via the FWHT, and $O(d^2 \log d)$ for the full matrix.

2. Compute the spectral energy at each degree: $E_i(k) = \sum_{|S|=k} \widehat{w}_i(S)^2$.

3. Compute the effective spectral degree k_i^* (95% energy threshold) for each row.

4. Aggregate: report the distribution of k^* across rows and layers.

Step 3: Controls. Compute the same metrics for:

(a) **Random binary matrices:** $W_{ij} \sim \text{Uniform}(\{-1, +1\})$. The expected spectral energy at degree k is $\binom{n}{k}/2^n$, following a $\text{Bin}(n, 1/2)$ distribution, so the 95%-energy threshold is $k^* \approx n/2 + O(\sqrt{n})$ with no low-degree concentration. Any concentration observed in trained weights significantly below this baseline is evidence of structure.

(b) **Random binary matrices with matched row-sum distribution:** To control for the possibility that concentration is merely an artifact of biased rows.

(c) **Random linear threshold functions (LTFs):** For each row, generate a random weight vector $a \sim \mathcal{N}(0, I_n)$ (where $n = \log_2 d$) and set $w(j) = \text{sgn}(\langle a, \text{bin}(j)\rangle)$. This produces binary vectors with the spectral profile characteristic of LTFs (strong degree-1, decaying higher degrees). If trained weights look like random LTFs, then any observed concentration reflects the sign-activation structure rather than learned task-specific structure.

(d) **Random ternary matrices with matched sparsity:** For the BitNet b1.58 analysis (which uses ternary $\{-1, 0, +1\}$ weights), generate random ternary matrices with the same fraction of zeros per row as the real weights. This controls for the possibility that spectral concentration is an artifact of zero-inflation rather than learned structure.

Step 4: Test column-indexing sensitivity. For each matrix, repeat the spectral analysis under 10 random bit permutations. Report whether the degree profile $\sum_{|S|=k} \widehat{w}(S)^2$ is stable (as theory predicts for bit permutations) and under 10 random column permutations (where the profile may change).

5.2 Expected Outcomes and Implications

Strong concentration (median $k^* \leq 3$ for $n = 10$, and significantly below the random-LTF control with $p < 0.01$ via a two-sample Kolmogorov–Smirnov test): This validates Conjecture 3.7 and strongly motivates the full spectral training program with degree truncation $k \leq 3$. **Action:** proceed to spectral training (Phase 3 of the work plan).

Moderate concentration (median $k^* \leq n/2$, with the k^* distribution shifted significantly below both the random-binary and random-LTF controls): The spectral parameterization may still offer benefits, but higher degree truncation or the Hadamard-structured approach may be needed. **Action:** proceed with both spectral and Hadamard approaches in parallel.

No concentration (median k^* statistically indistinguishable from random-LTF control, or $k^* > n/2$): This falsifies Conjecture 3.7 in its strong form. **Action:** pivot to the Hadamard-structured approach (Section 4.4), which does not depend on spectral sparsity; investigate whether concentration emerges during spectral *training* even if absent in conventionally trained weights; reframe the thesis contribution around the framework and diagnostics.

5.3 Computational Cost

The pilot study is computationally modest. For a model with $d = 1024$ and 12 layers, each layer has 6 weight matrices: W_Q, W_K, W_V, W_O (attention) plus $W_{\text{up}}, W_{\text{down}}$ (FFN), giving 72 matrices total. Note that the FFN matrices are not square ($d \times 4d$ and $4d \times d$ in a standard Transformer), so each row is analyzed at its actual length. The FWHT computation for all rows is $O(d \cdot d_{\text{row}} \log d_{\text{row}})$ per matrix. Including permutation-sensitivity tests and controls, we estimate 1–6 CPU-hours total depending on threading and the number of permutation repeats. The entire pilot study can be completed in a day.

6 Experimental Plan

6.1 Datasets and Tasks

Primary task. Autoregressive language modeling on WikiText-103 [Merity et al., 2017] and/or a subset of The Pile [Gao et al., 2021a]. Larger-scale runs (10–100B tokens from RedPajama [Together Computer, 2023]) if compute permits.

Auxiliary tasks. Synthetic Boolean function learning (majority, tribes, address function) and character-level language modeling (Penn Treebank, enwik8).

6.2 Baselines

1. **BitNet b1.58** [Ma et al., 2024b]: ternary weights, learned scaling factors, 8-bit activations.
2. **Standard binary STE**: binary $\{-1, +1\}$ weights, weight-domain STE, learned scaling factors.
3. **Full-precision Transformer**: FP16, same architecture. Upper bound.
4. **Random binary (frozen)**: random ± 1 weights, never updated. Lower bound.

6.3 Metrics

Primary. Perplexity: $\mathrm{PPL} = \exp(\mathcal{L}_{\mathrm{CE}})$.

Secondary. Zero-shot accuracy on LAMBADA, HellaSwag, ARC, PIQA, WinoGrande via lm-evaluation-harness [Gao et al., 2021b].

Efficiency. Training FLOPs, inference latency on CPU, model size in bits.

Spectral diagnostics.

(i) Effective spectral degree k^* (95% energy threshold), per layer, over training.

(ii) Spectral entropy: $H_{\mathrm{spec}} = -\sum_S p(S) \log p(S)$ where $p(S) = \widehat{w}(S)^2 / \sum_{S'} \widehat{w}(S')^2$.

(iii) Rounding error: $\varepsilon_{\mathrm{round}} = d^{-2} \sum_{i,j} (\mathrm{sgn}(z_i(j)) - z_i(j))^2$.

6.4 Ablation Studies

A1: Spectral degree k. Vary k from 1 to $n = \log_2 d$.

A2: Projection method. Hard sign vs. randomized rounding vs. spectral norm projection.

A3: Hadamard structure vs. unstructured.

A4: Spectral regularization. Degree penalty and noise stability reward.

A5: Activation precision. Binary (sgn) vs. 8-bit vs. full-precision activations.

A6: Embedding strategy. Learned FP, learned binary, fixed Hadamard.

A7: Sinkhorn rounding sensitivity. For Hadamard-structured layers, measure performance degradation when soft Sinkhorn permutations are rounded to hard permutations at various stages of training.

A8: Column encoding. Standard binary vs. Gray code vs. learned permutation (Section 4.2.2).

6.5 Implementation Roadmap

`SpectralLinear.` Drop-in for `nn.Linear`. State: spectral coefficients [`d_out`, $\binom{n}{\leq k}$], precomputed Walsh basis [$\binom{n}{\leq k}$, `d_in`]. Forward: $W = \mathrm{sgn}(\widehat{w} \cdot \Phi^\top) \cdot x$.

`HadamardLinear.` Structured factorization. State: L_H sign vectors, L_H permutation parameters. Forward: cascade of FWHT + sign-flip + permute.

`SpectralTransformerBlock.` Wraps the above into a Transformer block with attention and normalization.

`SpectralDiagnostics.` Computes all spectral metrics; logs histograms; detects anomalies.

`PilotAnalysis.` Scripts for the pilot study (Section 5): load pretrained weights, compute WHT, aggregate statistics, generate plots.

6.6 Success Criteria

Strong success. SBT achieves perplexity within 10% of BitNet b1.58, with strictly 1-bit weights and no learned scaling.

Moderate success. SBT outperforms Baseline 2 (weight-domain STE) and shows clear spectral concentration.

Diagnostic success. Spectral diagnostics reveal structure ($k^* \ll n$, decreasing spectral entropy).

Negative result. No spectral concentration and no performance gain. Informative and publishable.

6.7 Known Risks and Mitigation

Risk 1: Rounding destroys information. *Mitigation:* Temperature-annealed $\tanh(\beta z)$.

Risk 2: Spectral degree too high. *Mitigation:* Pilot study diagnoses this; Hadamard layers as fallback.

Risk 3: Training instability. *Mitigation:* Spectral norm projection; degree regularization; careful warmup.

Risk 4: Computational overhead. *Mitigation:* FWHT is $O(d \log d)$, negligible vs. $O(d^2)$ matmuls.

Risk 5: Column-indexing sensitivity. *Mitigation:* Pilot study tests this; Gray code and learned encodings as alternatives.

References

Josh Alman. Faster walsh-hadamard transform and matrix multiplication over finite fields using lookup tables, 2022. URL https://arxiv.org/abs/2211.04643.

William Beckner. Inequalities in Fourier analysis. *Annals of Mathematics*, 102(1):159–182, 1975.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013. URL https://arxiv.org/abs/1308.3432.

Aline Bonami. Étude des coefficients de Fourier des fonctions de $L^p(G)$. *Annales de l'Institut Fourier (Grenoble)*, 20(2):335–402, 1970. Published 1971.

Giorgio M. Cavallazzi, Miguel Pérez Cuadrado, and Alfredo Pinelli. Walsh-hadamard neural operators for solving pdes with discontinuous coefficients, 2025. URL https://arxiv.org/abs/2511.07347.

Krzysztof M. Choromanski, Mark Rowland, and Adrian Weller. The unreasonable effectiveness of structured random orthogonal embeddings. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://papers.nips.cc/paper/2017/hash/bf8229696f7a3bb4700cfddef19fa23f-Abstract.html.

Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, David Benjamin Belanger, Lucy J. Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=Ua6zuk0WRH.

Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. Binaryconnect: Training deep neural networks with binary weights during propagations. In *Advances in Neural Information Processing Systems*, volume 28, pages 3123–3131. Curran Associates, Inc., 2015.

Tri Dao, Albert Gu, Matthew Eichhorn, Atri Rudra, and Christopher Ré. Learning fast algorithms for linear transforms using butterfly factorizations. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 1517–1527. PMLR, 09–15 Jun 2019.

Tri Dao, Beidi Chen, Nimit S. Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive struc-

tured matrices for efficient and accurate training. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 4690–4721. PMLR, 17–23 Jul 2022.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *CoRR*, abs/2101.00027, 2021a. URL `https://arxiv.org/abs/2101.00027`.

Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Eric Hallahan, Horace He, Haonan Li, Kyle McDonell, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation. lm-evaluation-harness, 2021b. URL `https://github.com/EleutherAI/lm-evaluation-harness`.

Prahladh Harsha, Adam R. Klivans, and Raghu Meka. Bounding the sensitivity of polynomial threshold functions. *Theory of Computing*, 10:1–26, 2014. doi: 10.4086/toc.2014.v010a001.

Koen Helwegen, James Widdicombe, Luca Geiger, Zechun Liu, Kwang-Ting Cheng, and Roeland Nusselder. Latent weights do not exist: Rethinking binarized neural network optimization. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. In *NIPS*, pages 4107–4115, 2016.

Jeff Kahn, Gil Kalai, and Nathan Linial. The influence of variables on boolean functions (extended abstract). In *FOCS*, pages 68–80. IEEE Computer Society, 1988.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *ICML*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020.

Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization. *CoRR*, abs/2306.07629, 2023. URL `https://arxiv.org/abs/2306.07629`.

Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *ICLR*. OpenReview.net, 2020.

Eyal Kushilevitz and Yishay Mansour. Learning decision trees using the fourier spectrum (extended abstract). In *STOC*, pages 455–464. ACM, 1991.

Quoc V. Le, Tamas Sarlós, and Alexander J. Smola. Fastfood - approximating kernel expansions in loglinear time. In *Proceedings of the 30th International Conference on Machine Learning, ICML 2013, Atlanta, GA, USA, 16-21 June 2013*, volume 28 of *JMLR Workshop and Conference Proceedings*, pages 244–252. JMLR.org, 2013. URL `http://proceedings.mlr.press/v28/le13.html`.

Nathan Linial, Yishay Mansour, and Noam Nisan. Constant depth circuits, fourier transform and learnability. In *30th Annual Symposium on Foundations of Computer Science, FOCS 1989, Research Triangle Park, North Carolina, USA, 30 October - 1 November 1989*, pages 574–579. IEEE Computer Society, 1989. doi: 10.1109/SFCS.1989.63527.

Zechun Liu, Baoyuan Wu, Wenhan Luo, Xin Yang, Wei Liu, and Kwang-Ting Cheng. Bi-real net: Enhancing the performance of 1-bit CNNs with improved representational capability and advanced training algorithm. In *Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part X*, volume 11214 of *Lecture Notes in Computer Science*, pages 747–763. Springer, 2018. doi: 10.1007/978-3-030-01249-6_45.

Liqun Ma, Mingjie Sun, and Zhiqiang Shen. FBI-LLM: Scaling up fully binarized LLMs from scratch via autoregressive distillation. *CoRR*, abs/2407.07093, 2024a. URL `https://arxiv.org/abs/2407.07093`.

Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *CoRR*, abs/2402.17764, 2024b. URL `https://arxiv.org/abs/2402.17764`.

Yishay Mansour. Learning boolean functions via the fourier transform. In *Theoretical Advances in Neural Computation and Learning*, volume 275 of *The Springer International Series in Engineering and Computer Science*, pages 391–424. Springer, 1994. doi: 10.1007/978-1-4615-2686-6_15.

Gonzalo E. Mena, David Belanger, Scott Linderman, and Jasper Snoek. Learning latent permutations with Gumbel-Sinkhorn networks. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. URL `https://openreview.net/forum?id=Byt3vJeeZ`.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *5th International Conference on Learning Representations,*

ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings. OpenReview.net, 2017. URL `https://openreview.net/forum?id=Byj72udxe`. Introduces the WikiText dataset.

Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. Norm-based capacity control in neural networks. In *Proceedings of The 28th Conference on Learning Theory, COLT 2015, Paris, France, July 3-6, 2015*, volume 40 of *JMLR Workshop and Conference Proceedings*, pages 1376–1401. JMLR.org, 2015. URL `http://proceedings.mlr.press/v40/Neyshabur15.html`.

Ryan O'Donnell. *Analysis of Boolean Functions*. Cambridge University Press, 2014. ISBN 978-1-107-03832-5. doi: 10.1017/CBO9781139814782.

Hongyi Pan, Diaa Badawi, and Ahmet Enis Çetin. Fast walsh-hadamard transform and smooth-thresholding based binary layers in deep neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops 2021, BiVision*, pages 3697–3706, 2021. doi: 10.1109/CVPRW53098.2021.00410. URL `https://openaccess.thecvf.com/content/CVPR2021W/BiVision/html/Pan_Fast_Walsh-Hadamard_Transform_and_Smooth-Thresholding_Based_Binary_Layers_in_Deep_CVPRW_2021_paper.html`.

Hongyi Pan, Diaa Badawi, and Ahmet Enis Çetin. Block walsh-hadamard transform based binary layers in deep neural networks. *ACM Trans. Embedded Comput. Syst.*, 21(4):1–24, 2022. doi: 10.1145/3510830. URL `https://doi.org/10.1145/3510830`.

David A. Patterson, Joseph Gonzalez, Quoc V. Le, Chen Liang, Lluís-Miquel Munguía, Daniel Rothchild, David R. So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint*, 2021. URL `https://arxiv.org/abs/2104.10350`.

Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron C. Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019*, volume 97 of *Proceedings of Machine Learning Research*, pages 5301–5310. PMLR, 2019. URL `http://proceedings.mlr.press/v97/rahaman19a.html`.

Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. XNOR-Net: Imagenet classification using binary convolutional neural networks. In *Computer Vision - ECCV 2016 - 14th European Conference*, volume 9908 of *Lecture Notes in Computer Science*, pages 525–542. Springer, 2016. doi: 10.1007/978-3-319-46493-0_32.

Netanel Raviv, Aidan Kelley, Minzhe Guo, and Yevgeniy Vorobeychik. Enhancing robustness of neural networks through fourier stabilization. In *Proceedings of the*

38th International Conference on Machine Learning, ICML 2021, volume 139 of *Proceedings of Machine Learning Research*, pages 8880–8889. PMLR, 2021. URL http://proceedings.mlr.press/v139/raviv21a.html.

Together Computer. Redpajama: An open dataset for training large language models. Dataset and technical report, 2023. URL https://github.com/togethercomputer/redpajama-data. Released April 2023. A 1.2 trillion token open-source reproduction of the LLaMA training dataset.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint*, 2023. URL https://arxiv.org/abs/2310.11453.

Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley J. Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets. In *7th International Conference on Learning Representations, ICLR 2019*. OpenReview.net, 2019. URL https://openreview.net/forum?id=Skh4jRcKQ.

Shuchang Zhou, Zekun Ni, Xinyu Zhou, He Wen, Yuxin Wu, and Yuheng Zou. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv preprint*, 2016. URL https://arxiv.org/abs/1606.06160.